WEEK 4 PROJECT REPORT

AI AGENT SYSTEMS

Retrieval-Augmented Generation (RAG) using LangChain, LangGraph and FAISS

Student Name: Lokesh Thorat

Internship/Program: Skill Set Go EduTech

Project Period: Week 4

1. Abstract

This project implements a Retrieval-Augmented Generation (RAG) workflow that answers questions using a small AI/ML notes document as its knowledge source. The document is split into chunks, converted into embeddings, and stored in a FAISS vector store. For each question, the system retrieves relevant text and passes the question and context through a LangGraph workflow. A local FLAN-T5 model is used for answer generation, avoiding dependence on a paid hosted language-model API.

2. Problem Statement

A general language model may answer without using a specific supplied document. The goal of this project is to create a simple question-answering system that retrieves relevant information from a local knowledge source and uses that context to formulate answers.

3. Objectives

- Prepare a small AI/ML notes document as the knowledge base.
- Split the document into smaller text chunks.
- Create text embeddings and index them with FAISS.
- Retrieve relevant chunks for a user's question.
- Connect retrieval and answer generation in a LangGraph workflow.
- Test the workflow on questions from several AI/ML topics and record evaluation results.

4. Tools and Technologies

Tool / Technology	Purpose
Python / Google Colab	Implementation and execution environment
LangChain	Document processing and retrieval components
RecursiveCharacterTextSplitter	Splitting source notes into manageable chunks
Sentence Transformers: all-MiniLM-L6-v2	Creating text embeddings
FAISS	Vector similarity search over embedded chunks
LangGraph	Connecting retrieval and answer-generation steps
Hugging Face Transformers / FLAN-T5	Local text generation
LangSmith	Tracing and observability setup; traces were not confirmed

5. System Architecture

User Question → Text Embedding / Similarity Search → FAISS Retriever → Retrieved Context + Question → LangGraph Answer Node → Final Answer

The source document is first split into chunks. Each chunk is embedded and stored in FAISS. At query time, the retriever finds relevant chunks. The LangGraph workflow passes the retrieved context to the answer node, which uses the local FLAN-T5 model or the notebook's configured fallback logic to produce the response.

6. Implementation Steps

1. **Knowledge source:** Created an AI/ML notes text file containing short definitions and applications.
2. **Chunking:** Used RecursiveCharacterTextSplitter with chunk_size=250 and chunk_overlap=40.
3. **Embeddings:** Loaded sentence-transformers/all-MiniLM-L6-v2 to convert text chunks into vectors.
4. **Vector database:** Built a FAISS vector store to support similarity-based retrieval.
5. **RAG baseline:** Tested retrieval and a baseline response using retrieved context.
6. **Agent workflow:** Created a LangGraph workflow with retrieval and answer-generation nodes.
7. **Local generation:** Loaded FLAN-T5 small locally after the hosted OpenAI API returned an insufficientquota error.
8. **Testing:** Ran six questions covering AI, ML, Deep Learning, NLP, Computer Vision, and AI applications.

7. Testing and Results

The workflow was tested with six questions related to core AI/ML concepts. The notebook produced non-empty responses for all six test questions, giving an answer-generation rate of 6/6 (100%). This is a completion measure, not a verified answer-accuracy score.

Topic	Expected subject	Observed result
Artificial Intelligence	Meaning of AI	A definition of AI was returned
Machine Learning	Learning patterns from data	A definition of ML was returned
Deep Learning	Multi-layer neural networks	A definition of DL was returned
Natural Language Processing	Computers process human language	A rule-based fallback response was used
Computer Vision	Understanding images and videos	A definition of Computer Vision was returned
AI applications	Examples of real-world uses	Several application areas were listed

8. Evaluation

The notebook saved results to rag_evaluation_results.csv. The file includes recorded answers and text-similarity comparisons against expected answers. Text similarity is a rough lexical comparison and must not be interpreted as factual accuracy. The NLP response in the current notebook uses a hard-coded fallback because the local model returned an insufficiently informative output for that query.

LangSmith tracing was configured and a test was attempted, but traces did not appear in the project dashboard during verification. Therefore, LangSmith tracing is listed as pending rather than completed.

9. Limitations

- The knowledge base is small and covers only the content included in the notes file.

- A small local model can produce short, incomplete, or inaccurate responses.
- The evaluation uses a small set of six questions.
- Text similarity does not establish factual correctness.
- LangSmith traces were not confirmed in the dashboard.

10. Future Improvements

- Expand the knowledge base with more reliable documents.
- Improve answer prompting and inspect retrieved chunks for every test query.
- Evaluate answers with a larger test set and human-reviewed correctness criteria.
- Add a user interface for asking questions.
- Resolve and verify LangSmith tracing before claiming observability results.

11. Conclusion

This project demonstrates the main components of a basic RAG agent system: document chunking, embedding generation, FAISS retrieval, and a LangGraph workflow connected to local text generation. The system returned responses for six test questions and exported evaluation results to CSV. The work provides a foundation for building more capable document-grounded AI assistants, while highlighting the need for stronger answer validation and verified tracing.

12. Repository Files

Week4_AI_Agent_Systems.ipynb — notebook implementation

rag_evaluation_results.csv — recorded evaluation results

README.md — project overview and usage summary

.